

Descrição do problema

Observando o funcionamento do colégio sem um sistema de *Help Desk* para integrar professores e a equipe de TI, são notórios os problemas que se acumulam como efeito dessa carência.

Primeiramente, o mais notável entre esses obstáculos é a desorganização na questão da priorização. Sem um sistema que separe os pedidos utilizando o critério de urgência, a equipe atende os pedidos sem uma ordem clara, neste cenário, uma falha trivial pode ser priorizada em detrimento de uma crucial. Em decorrência dessa desordem, as aulas são diretamente impactadas, quando há algum problema com os eletrônicos em sala, o docente não sabe se e em quanto tempo será atendido, comprometendo o planejamento pedagógico.

Em segundo lugar, a falta de um sistema que organize os chamados dos docentes para a equipe de TI causa grande dificuldade de gestão e planejamento. Nesse contexto, a falta de registros formais dos impasses encontrados pelos professores faz com que a equipe não saiba quantos problemas existem, quais já foram solucionados e quais foram esquecidos. Além disso, sem esses dados, a equipe não consegue analisá-los e identificar padrões, levando a correções pontuais dos sintomas do problema ao invés de identificar a causa. Ademais, a equipe gestora fica desorientada e encontra dificuldades em dimensionar a equipe e tomar decisões relacionadas à compra de novos equipamentos.

Por fim, como consequência dos outros desafios, a equipe de TI enfrenta grande sobrecarga. Interrupções constantes por pedidos informais fragmentam o trabalho, desorganizando o cronograma já previsto, deixando pouco tempo para manutenções preventivas, atualizações e outros projetos de melhorias.

Portanto, à partir da análise da situação atual do funcionamento dos pedidos de reparos, faz-se necessário um sistema de *Help Desk* que integre a comunicação entre as duas partes do pedido e colete dados para melhor análise e maior eficiência.

Entrevista semiestruturada com um colaborador da escola

Para compreender melhor as necessidades relacionadas ao atendimento de solicitações de suporte técnico na instituição, foi realizada uma entrevista

semiestruturada com um colaborador do setor de Tecnologia da Informação (TI) da escola. A entrevista teve como objetivo levantar informações sobre o funcionamento atual do suporte, os principais tipos de solicitações recebidas pela equipe, as dificuldades encontradas no gerenciamento dos chamados e as características que poderiam ser aprimoradas em uma nova solução de helpdesk.

Durante a entrevista, foi identificado que a escola já utiliza atualmente um sistema destinado ao registro e gerenciamento de chamados de suporte técnico. A existência dessa ferramenta demonstra que a instituição já possui um processo estabelecido para encaminhar solicitações à equipe de TI. Entretanto, conforme relatado pelo entrevistado, o sistema utilizado apresenta algumas limitações que acabam dificultando sua utilização no dia a dia. Apesar de cumprir a função básica de registrar as solicitações, a plataforma não proporciona uma experiência considerada suficientemente prática e intuitiva para os usuários e para os próprios responsáveis pelo atendimento.

Entre as principais dificuldades observadas está a complexidade de utilização do sistema. O processo para registrar uma solicitação pode não ser suficientemente claro para usuários que não possuem familiaridade com ferramentas de suporte técnico, o que pode fazer com que algumas pessoas tenham dificuldade para identificar onde e como devem registrar seus problemas. Além disso, a organização das informações e o acompanhamento dos chamados podem tornar o processo menos eficiente, principalmente quando existe um número elevado de solicitações sendo atendidas simultaneamente.

Outro aspecto levantado durante a entrevista foi a necessidade de melhorar a comunicação entre o usuário e a equipe de TI. Em um ambiente escolar, diferentes pessoas podem precisar de suporte diariamente, incluindo alunos, professores e funcionários. As solicitações podem envolver problemas relacionados a computadores, acesso a sistemas, internet, equipamentos, contas institucionais, projetores, impressoras e outros recursos tecnológicos utilizados nas atividades da escola. Dessa forma, uma ferramenta de atendimento precisa permitir que essas demandas sejam registradas de maneira clara e que a equipe responsável consiga visualizar, organizar, priorizar e acompanhar cada solicitação de forma adequada.

A partir do relato do entrevistado, percebeu-se também que a eficiência de um sistema de helpdesk não depende apenas da possibilidade de abrir chamados, mas também da maneira como as informações são apresentadas e administradas

ao longo do processo de atendimento. Um sistema pouco intuitivo pode gerar dificuldades tanto para quem solicita o suporte quanto para quem precisa solucionar o problema. Quando o usuário não consegue registrar corretamente uma solicitação ou não encontra informações sobre o andamento de seu chamado, podem ocorrer dúvidas, solicitações repetidas e dificuldades na comunicação com a equipe de TI. Da mesma forma, quando a equipe possui dificuldades para visualizar e organizar os chamados, o gerenciamento das demandas pode se tornar mais trabalhoso.

Nesse contexto, a entrevista contribuiu para identificar uma oportunidade de melhoria no processo existente. A proposta do projeto não consiste apenas em substituir o sistema atualmente utilizado, mas em desenvolver uma solução que considere as dificuldades observadas pelos profissionais que utilizam e administram esse processo. A intenção é criar uma plataforma com uma interface mais simples e intuitiva, permitindo que o usuário consiga registrar sua solicitação de maneira objetiva e acompanhar seu atendimento sem precisar lidar com processos desnecessariamente complexos.

Além da facilidade de utilização, as informações obtidas durante a entrevista serviram como base para definir algumas das funcionalidades consideradas relevantes para o novo sistema. Entre elas, destaca-se a possibilidade de registrar chamados contendo informações sobre o problema, acompanhar seu status, visualizar o histórico das solicitações e permitir que a equipe de TI organize os atendimentos de acordo com suas respectivas situações e prioridades. Dessa forma, pretende-se centralizar as informações relacionadas ao suporte em um único ambiente, facilitando tanto a comunicação com os usuários quanto o gerenciamento interno das demandas.

A entrevista também permitiu compreender que o desenvolvimento da nova plataforma deve considerar diferentes perfis de usuários. Enquanto uma pessoa que solicita suporte precisa de uma interface simples, com informações claras e poucos passos para registrar um problema, a equipe de TI necessita de recursos que possibilitem administrar os chamados, atualizar seus status, visualizar as demandas pendentes e manter um histórico dos atendimentos realizados. Portanto, o sistema proposto deve atender às necessidades de ambos os lados do processo.

Com base nas informações coletadas, foi possível estabelecer que um dos principais problemas a serem considerados no desenvolvimento do projeto é a dificuldade de utilização e a baixa praticidade percebida no sistema atualmente

empregado pela escola. A entrevista, portanto, teve papel importante na identificação dos requisitos da nova solução, permitindo que o desenvolvimento fosse orientado não apenas por aspectos técnicos, mas também pelas necessidades reais observadas no ambiente escolar. As informações obtidas serão utilizadas como referência para as etapas seguintes do projeto, especialmente na definição dos requisitos funcionais, na elaboração da interface e na organização do fluxo de atendimento dos chamados.

Como funciona o sistema

Estrutura de pastas

HELPDESK-BQ-MAIN/

```
|— db/
|   |— migrations/
|— docs/
|— middlewares/
|— public/
|— src/
|   |— controllers/
|   |— repositories/
|   |— routes/
|   |— services/
|— tests/
|— app.js
|— server.js
|— .gitignore
|— README.md
```

Funcionalidades

db/migrations/	Armazena as migrações responsáveis pela criação e evolução do banco de dados.
docs/	Guarda a documentação, decisões técnicas, contratos e demais

	documentos do projeto.
middlewares/	Contém funções intermediárias utilizadas durante o processamento das requisições, como autenticação e tratamento de erros.
public/	Contém os arquivos da interface web, como HTML, CSS e JavaScript.
src/routes/	Define as rotas e endpoints disponibilizados pela API.
src/controllers/	Recebe as requisições, chama os serviços necessários e prepara as respostas HTTP.
src/services/	Concentra as regras de negócio, permissões e transições de estado dos chamados.
src/repositories/	Responsável pelo acesso e pelas consultas ao banco de dados.
tests/	Contém os testes automatizados do sistema.
app.js	Configura a aplicação Express e suas rotas e middlewares.
server.js	Inicializa o servidor e a porta de execução.
README.md	Apresenta informações gerais do projeto e instruções para sua execução.

Fluxo de funcionamento

Usuário → Interface web → Rotas da API → Controllers → Services → Repositories → Banco de dados

Por exemplo, quando um usuário cria um chamado, a interface envia os dados para a API. A rota recebe a requisição, o controller coordena o processamento, o service verifica as regras de negócio e as permissões e o repository realiza a operação necessária no banco.

Telas e suas funções

O sistema é composto pelas telas abaixo. Existem cinco principais: Login, Lista de chamados, Novo chamado, Detalhe do chamado e Dashboard.

1. Tela de Login

É a porta de entrada do sistema.

Nessa tela, o usuário informa:

- e-mail;
- senha;
- ação de acesso;
- mensagens de erro ou bloqueio.

O sistema deve autenticar o usuário e impedir acesso quando as credenciais forem inválidas. Após três tentativas inválidas dentro de cinco minutos, a conta deve permanecer bloqueada durante dez minutos.

2. Lista de chamados

Apresenta os chamados disponíveis para o usuário de acordo com suas permissões.

As principais funcionalidades são:

- visualizar chamados;
- filtrar por status;
- filtrar por prioridade;
- combinar filtros;
- navegar pelos resultados;
- apresentar estado vazio quando não houver registros.

O solicitante visualiza seus próprios chamados, enquanto o técnico visualiza a fila para a qual possui autorização.

3. Tela de Novo Chamado

Permite registrar uma nova solicitação.

O formulário possui:

- título;
- descrição;
- prioridade.

O título deve possuir entre 10 e 100 caracteres, enquanto a descrição deve possuir pelo menos 30 caracteres. A prioridade pode ser baixa, média, alta ou crítica.

A tela também deve apresentar mensagens de validação e tratar situações como perda de conexão durante o envio.

4. Tela de Detalhes do Chamado

Exibe todas as informações relevantes de um chamado e permite acompanhar seu atendimento.

Entre as funcionalidades estão:

- visualizar os dados do chamado;
- consultar o histórico;
- adicionar comentários;
- alterar status;
- alterar prioridade, conforme permissão;
- executar ações permitidas ao perfil do usuário.

As alterações devem ser registradas no histórico com informações como autor, data, ação, valor anterior e novo valor.

Os status seguem uma sequência controlada: Aberto -> Em análise -> Em atendimento -> Resolvido -> Fechado

ambém existe a possibilidade de retornar de **resolvido para em atendimento**. Um chamado crítico não pode ser fechado sem um comentário de resolução.

5. Dashboard / Relatórios

Apresenta uma visão resumida dos chamados.

O dashboard permite consultar:

- quantidade de chamados por status;
- quantidade por prioridade;
- filtros;
- período de consulta;
- estado vazio quando não existem dados.

A API prevê o endpoint de resumo `/api/reports/summary`, responsável por retornar esses totais ou informar corretamente a ausência de dados.

Funcionalidades por tipo de usuário

O sistema possui três tipos principais de usuário:

Usuário comum / solicitante	Criar e acompanhar seus chamados.
Técnico	Consultar sua fila, priorizar, comentar e alterar o status dos chamados autorizados.
Administrador	Administrar permissões e categorias e executar ações administrativas, que devem ser registradas no histórico.

Como será o desenvolvimento

Visão geral e tecnologias

O sistema será desenvolvido como uma aplicação web com API própria, organizada em camadas. O backend será construído em **JavaScript com Node.js** e o framework **Express**, enquanto o frontend será feito com **HTML, CSS e JavaScript** servidos pela própria aplicação, a partir da pasta `public/`. A escolha por uma única linguagem nas duas pontas simplifica a comunicação entre as equipes e reduz a curva de aprendizado do grupo.

Camada	Tecnologia
- Linguagem	- JavaScript (Node.js)
- Backend / API	- Express
- Frontend	- HTML, CSS e JavaScript
- Banco de dados	- PostgreSQL, com migrações em db/migrations/
- Autenticação	- JWT para sessão e bcrypt para criptografia de senhas
- Testes automatizados	- Jest e Supertest
- Controle de versão	- Git e GitHub

Arquitetura

O backend segue o fluxo **Interface web** → **Rotas** → **Controllers** → **Services** → **Repositories** → **Banco de dados**. Cada camada tem uma responsabilidade única: as rotas definem os endpoints; os controllers recebem a requisição e preparam a resposta HTTP; os services concentram as regras de negócio, permissões e transições de status; e os repositories fazem o acesso ao banco. Essa separação permite que cada integrante trabalhe em uma parte sem interferir nas demais e facilita a criação de testes isolados.

Divisão do trabalho

Backend (**src/**, **middlewares/**, **db/**)

- Configurar o Express (**app.js**) e o servidor (**server.js**).
- Criar as migrações do banco: usuários, chamados, comentários, histórico e categorias.
- Implementar login com bloqueio de 10 minutos após três tentativas inválidas em cinco minutos.
- Implementar o CRUD de chamados, com as validações de título (10 a 100 caracteres), descrição (mínimo de 30 caracteres) e prioridade (baixa, média, alta ou crítica).
- Implementar o controle de status (Aberto → Em análise → Em atendimento → Resolvido → Fechado), o retorno de Resolvido para Em atendimento e a regra de que chamado crítico só fecha com comentário de resolução.
- Registrar no histórico autor, data, ação, valor anterior e novo valor.
- Controlar permissões por perfil (solicitante, técnico e administrador) em middlewares.
- Disponibilizar o endpoint **/api/reports/summary** para o dashboard.

Frontend (**public/**)

- Desenvolver as cinco telas: Login, Lista de chamados, Novo chamado, Detalhes do chamado e Dashboard.
- Implementar filtros por status e prioridade, estados vazios e mensagens de validação.
- Tratar falhas, como perda de conexão durante o envio de um chamado.
- Consumir a API do backend e manter uma interface simples e intuitiva, conforme apontado na entrevista.

Testes (**tests/**)

- Testes unitários dos services, cobrindo regras de negócio como transições de status e bloqueio de login.
- Testes de integração das rotas da API com Supertest.

- Testes funcionais das telas, validando fluxos completos como criar um chamado, filtrar a lista e fechar um chamado crítico.
- Registro dos casos de teste, com resultado esperado e obtido, na pasta [docs/](#).

Organização da equipe

O código ficará em um repositório no GitHub, com cada integrante trabalhando em branches próprias e integrando por *pull requests*. A pasta [docs/](#) reunirá a documentação e as decisões técnicas, e o README trará as instruções de instalação e execução.